大语言模型的训练

第 9 章（上）

第 13 课时 本课时讲解 9.1、9.2（前半）

训练显存与并行 · 混合并行策略 · 分布式训练系统

著者
邓杭　杨耀东

教材
《大语言模型：训练、精调与对齐》

出版
清华大学出版社



本章目录 第 13 课时

9.1 **大语言模型训练** 第 3 页
三大挑战、训练框架、显存开销与通信基础

9.2 **混合并行策略** 第 24 页
四种并行及其正交组合

9.3 **分布式训练系统** 第 61 页
DeepSpeed 零冗余优化器与显存分区

9.4 **小结** 第 75 页
并行算法的取舍与系统视角

9.5 **附录：专家并行*** 第 79 页
MoE 的专家部署与两次全交换

9.6 **附录：训练-推理解耦架构*** 第 84 页
RL 训练的采样部署、参数同步与异步推理

本课时内容 9.1 大语言模型训练、9.2 混合并行策略（前半）（其余部分见第 14 课时）

9.1

大语言模型训练

训练规模的增长把问题从“模型能不能学”推到“系统能不能撑住”：显存、带宽与可靠性共同决定训练能否跑完。

本节内容

9.1.1 介绍

9.1.2 分布式训练框架

9.1.3 训练目标

9.1.4 设备逻辑架构

9.1.5 分布式通信操作

大语言模型的训练规模

随着预训练模型的参数规模与训练语料库持续增长，模型训练对计算资源的需求急剧上升，通常需要庞大的 GPU 集群进行长时间持续训练。

16 000

H100-80GB GPU，Llama-3 预训练所用的集群规模

54

天，一次预训练持续的时间

3

核心挑战：可扩展性、高效性、可靠性

> **指标** 本章重点聚焦提升训练软件框架的高效性，其核心衡量指标是吞吐率：平均每秒每 GPU 处理的词元数量。该指标值越高，表明运行效率越高。

吞吐率是算法设计、计算优化、网络通信、存储管理和训练框架实现等多方面因素共同作用的结果，因此需要逐层拆解系统的瓶颈。

训练的三大挑战

大规模语言模型训练面临的核心挑战在于基础设施的可扩展性、高效性和可靠性。

挑战 1

可扩展性

要求基础设施和系统能够无缝扩展到更多 GPU 或其他计算设备上，同时保持训练的准确性和模型精度。

涉及分布式训练系统对硬件网络、存储和训练框架的协同设计。

挑战 2

高效性

旨在最大化集群资源的利用率，通常使用模型 FLOPs 利用率（MFU）作为衡量指标。

需要精心设计并行算法、优化计算资源利用、最小化通信开销，并实施高效的存储管理。

挑战 3

可靠性

源于预训练过程往往持续数周甚至数月，必须确保训练过程中硬件和软件的稳定性。

需要设计有效的容错机制，并在训练因故中断后能迅速恢复。

三个挑战相互牵制：并行度越高越容易扩展，但通信开销上升会压低 MFU；而规模越大、运行越久，故障概率也越高。

本章结构

预训练中常用的分布式训练策略是混合并行训练，其关键组成部分包括数据并行、张量并行、流水线并行和序列并行。

逻辑设备

9.1.4 节

集群的通信层级

通信操作

9.1.5 节

集合通信原语

混合并行

9.2 节

DP / TP / PP / CP

ZeRO

9.3 节

显存冗余消除

- DeepSpeed 框架中的 ZeRO 技术能显著提升集群上的存储效率，见 9.3 节。
- 对于集成了 MoE 的网络结构，还需引入专家并行，见 9.5 节附录。
- 基于强化学习的训练算法需要训练系统与推理系统并行部署，见 9.6 节附录。

依据 《大语言模型：训练、精调与对齐》第 9.1.1 节

效率优化技术

技术 1

梯度检查点

前向计算时仅保留部分激活值，反向传播时重新计算丢弃的部分，显著降低显存消耗。

技术 2

混合精度训练

结合使用不同精度的浮点数，在保持数值稳定的同时加速计算。

技术 3

低比特训练

使用 FP8 等超低精度格式，能极大减少计算量和通信量。

技术 4

计算内核优化

通过定制高性能计算内核（如融合算子）进一步提升训练速度。

DeepSeek-V3 的三个工程选择说明了这些技术的组合方式：双向流水线并行、计算-通信重叠掩盖专家并行的通信延迟，以及在其大型模型训练中应用的 8 位精度训练技术。

> **注** 计算-通信重叠是贯穿全章的主线：通信本身的耗时难以消除，但可以把通信藏在计算背后（详见 9.2 节的 DualPipe 与 9.5 节）。

分布式训练框架的分类

在大规模深度学习训练中，常用的分布式训练框架包括 PyTorch、DeepSpeed、Megatron-LM 和 Ray，它们各自负责训练栈的不同层次。

框架 1

PyTorch.Distributed

支持在多台机器和多个 GPU 上进行训练，提供通信操作接口和进程组管理功能，支持数据并行策略。

框架 2

DeepSpeed

分布式数据并行的优化实现，核心优势在于 ZeRO 优化器系列技术，可对参数、梯度、优化器状态分片或卸载。

框架 3

Megatron-LM

定制优化了 Transformer 架构，深度集成张量并行与流水线并行，通过算子切分规则与通信模式分布参数。

框架 4

Ray

通用的分布式计算框架，提供灵活的 API 与任务调度、资源管理，可协调训练与在线采样两类任务。

PyTorch.Distributed

PyTorch.Distributed 是 PyTorch 中用于支持分布式训练的模块，它通过抽象底层通信接口，支持多种后端，并通常基于单程序多数据（SPMD）范式进行编程。

- 提供通信操作接口和进程组管理功能，支持数据并行策略。
- 提供 All-Reduce、All-Gather、All-to-All 等通信原语，实现不同进程间的数据共享与同步。

SPMD

单程序多数据

所有进程执行同一份程序代码，只是各自处理不同的数据分片，并通过 rank 区分身份。

这种范式让并行算法的表达与设备数量解耦：同一段代码在 8 卡与 800 卡上都能运行，只需改变通信组的大小。

> **注** PyTorch.Distributed 只提供“通信怎么做”，不决定“如何切分模型”。张量并行、流水线并行的切分规则由 Megatron-LM 这类框架在其之上实现。

四种框架的定位

框架	主导方	核心机制	在训练栈中的位置
PyTorch.Distributed	PyTorch 社区	集合通信原语、进程组、SPMD	通信底座，数据并行的实现基础
DeepSpeed	微软	ZeRO：参数、梯度、优化器状态分片与卸载	数据并行的显存优化层
Megatron-LM	NVIDIA	张量并行、流水线并行的算子切分与通信模式	模型并行的实现层
Ray	加州大学伯克利分校	通用任务调度、资源管理、参数服务器	跨任务的编排层，用于 RL 训练与采样

四者的关系不是替代而是分工：Megatron-LM 解决“模型放不下”，DeepSpeed 解决“状态太冗余”，Ray 解决“任务怎么编排”，PyTorch.Distributed 提供底层的通信能力。

在 LLM 强化学习训练场景（如 PPO 和 GRPO）中，常需在线采样。Ray 能有效协调训练和在线采样任务，并通过参数服务器模式或共享存储机制管理参数状态、保障分布式环境下的同步一致性。

分布式训练框架概览

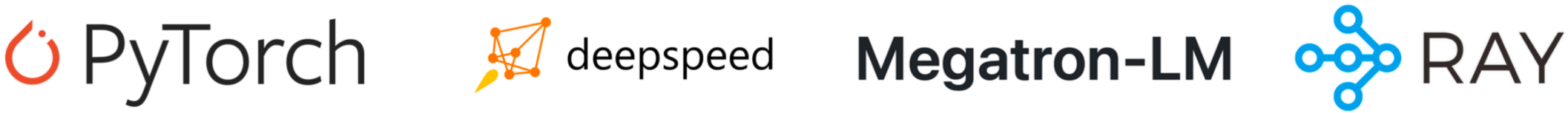


图 9.1　常用的分布式训练框架

框架之外还有两个补充工具，二者的共同思路都是用 Triton 重写关键算子。

算子层

Liger Kernel

针对大语言模型训练优化的 Triton 内核集合，重写 RMSNorm、RoPE、SwiGLU、CrossEntropy 等常见训练算子，显著提升多 GPU 训练的吞吐量并降低内存占用。

微调层

Unsloth

面向高效微调的开源框架，使用 Triton 手工重写关键 PyTorch 算子，把计算密集型操作直接优化为高性能 GPU 内核，消除计算冗余。

训练显存的组成

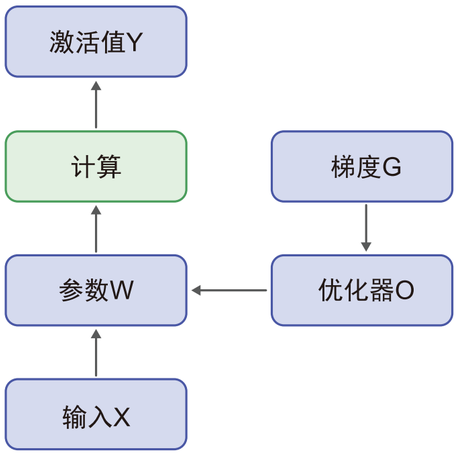


图 9.2　语言模型训练过程存储占用

主流大语言模型通常采用纯解码器架构的 Transformer 模型，例如 GPT 和 Llama 系列。训练时通常使用 Adam 优化器。

- 前向传播过程会保留**激活值**，用于反向传播时的梯度计算。
- Adam 优化器自身维护的**动量**和**方差**状态也会占用显存。
- 此外还需保存**模型参数**与**梯度**，四项共同构成常驻开销。

分布式的必要性

压力 1

显存放不下

训练所需显存通常远超单设备的存储容量，因此需要将数据或模型切分到多个设备上进行分布式训练。

压力 2

通信停不住

设备间需要通过通信操作同步参数或中间数据，这些通信可能导致计算单元空闲等待，降低训练效率。

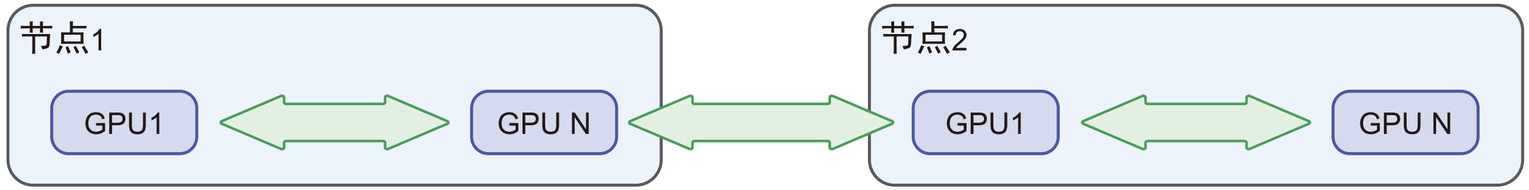


图 9.3　通信操作导致设备计算资源空闲，降低训练 MFU 效率

提升 MFU 有两条路：优化计算与通信的重叠；把数据分发到更多设备以提高并行度。通常采用混合并行策略，同时减少显存占用和通信开销。

分布式训练中的通信操作（如 All-Gather）有时也会产生需要临时存储的中间数据，这部分显存开销同样要计入预算。

集群的三级结构

在一个计算集群中，计算设备的最基本单元是 GPU。多个 GPU 组成一个计算节点，多个计算节点则构成整个集群；除计算设备外，网络组件也是集群的关键部分。

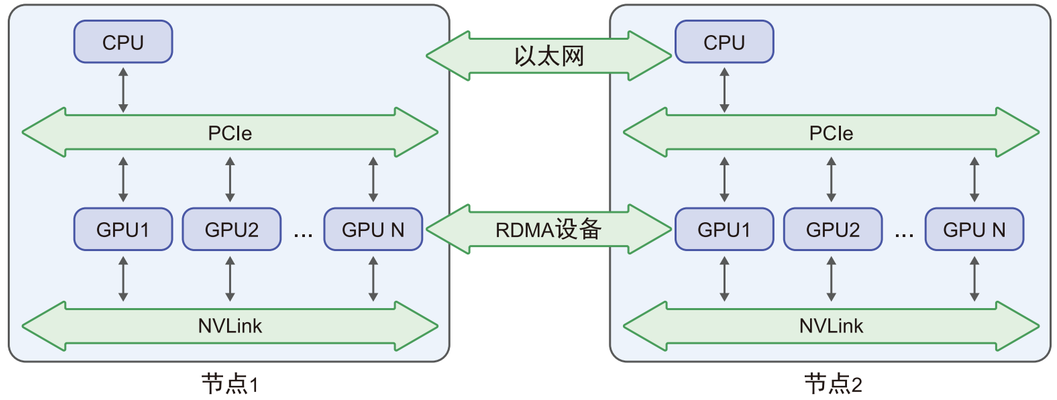


图 9.4　多节点多 GPU 集群设备间的通信逻辑架构图

GPU

计算的基本单元

→

节点

多块 GPU + 节点内互连

→

集群

多节点 + 节点间网络

三种通信链路

第 1 层 · CPU 与 GPU

PCIe 总线

数据或模型从存储设备传输到 GPU 即通过此通道。

这一层决定了数据加载的上限，数据管道供不上时 GPU 会空转。

第 2 层 · 节点内 GPU 与 GPU

PCIe / NVLink / NVSwitch

基础通信通过 PCIe 实现（如 GPU Direct P2P）；高速互连可采用 NVLink 提供高带宽、低延迟的点对点直接连接。

NVSwitch 作为交换芯片，为连接在其上的多个 GPU 提供全带宽、无阻塞的通信能力。

第 3 层 · 节点间 GPU 通信

RDMA

传统节点间通信基于以太网与 TCP/IP 协议栈，需要操作系统和 CPU 深度参与，延迟较高。

RDMA 允许一台计算机直接访问另一台计算机的内存，无需操作系统介入，显著降低延迟、提高吞吐量。

实现 RDMA 可通过专用的 InfiniBand 网络，或支持 RoCE（RDMA over Converged Ethernet）的以太网实现高速互联。

带宽的层级差异直接决定了并行算法的部署位置：通信密集型的任务（如张量并行）尽量限制在节点内完成；通信频率较低的任务（如数据并行，其主要通信发生在梯度同步阶段）则适合跨节点部署。

依据 《大语言模型：训练、精调与对齐》第 9.1.4 节

通信的初始化

网络拓扑

初始化 Mesh

通信群组

划分参与通信的进程

标识

本地设备号、IP 与端口

通信库

MPI / NCCL / GLOO

- 系统中的每个设备或进程拥有一个固定的本地设备号，并在网络层面通过唯一的 IP 地址和端口号进行标识。
- 通过通信库（如 MPI、NCCL 和 GLOO 等后端）初始化后，进程间可利用逻辑进程标识符（如 rank）或网络地址进行点对点通信。
- 点对点通信支持阻塞或非阻塞模式，是构造集合通信的基础。

rank 是并行算法中的“坐标”：数据切分、参数分区、环形拓扑中的前驱与后继，最终都要落到某个 rank 上。

阻塞通信与异步通信

阻塞式通信（同步通信）

要求发送方和接收方必须均完成数据发送和接收操作后，才能各自继续执行后续操作。

语义简单、易于推理，但设备在等待对端时完全空闲。

非阻塞式通信（异步通信）

允许发送方在发起发送操作后立即返回，继续执行其他计算任务，而无需等待接收方完成接收；接收方同样可立即返回，待数据传输完成后再处理。

异步通信有助于减少通信等待时间，提高计算资源利用率。

> **作用** 异步通信在异步数据并行和零气泡流水线并行等优化技术中起到关键作用。

本章后续的所有“重叠”技巧——无论是流水线的 1F1B、ZeroBubble 的 1F1B1W，还是 DeepSeek-V3 的 DualPipe——本质上都是把阻塞式通信改造成可与计算并行推进的异步通信。

集合通信原语

除点对点通信外，分布式训练广泛依赖一组标准的集合通信操作，包括广播、散播、全收集、全规约和全交换等。这些操作在通信群组内所有进程间协调执行，底层实现通常基于点对点通信原语，且是去中心化的。

操作	数据流向	典型用途
Broadcast	根进程的一份数据复制到全部设备	分发初始参数、配置
Scatter	根进程的 N 个子块分发给 N 个设备	切分数据或参数
Gather / All-Gather	各设备的数据块汇聚到根 / 汇聚到全部	拼回完整输出、ZeRO-3 取参数
Reduce / All-Reduce	规约结果存于根 / 全部设备各得一份	梯度同步
All-to-All	每台设备向每台设备发送不同的块	专家并行的派发与合并

熟练掌握这些常规集合通信操作，是实现更复杂分布式训练算法的基础：数据并行只用到 All-Reduce，ZeRO 用到 Reduce-Scatter 与 All-Gather，专家并行用到 All-to-All。

广播与散播

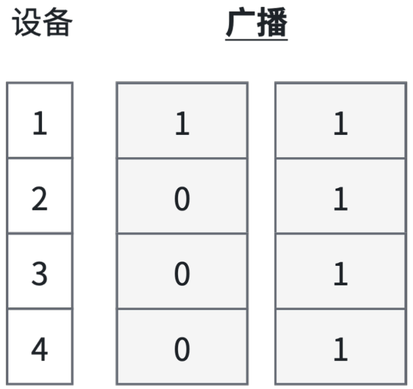


图 9.5(a)　广播



图 9.5(b)　散播

广播 Broadcast

设组内共有 N 个设备，根进程 i 将其数据 $\boldsymbol{X}$ 发送给其他 $N-1$ 个设备，最终所有设备均持有 $\boldsymbol{X}$ 的完整副本。

散播 Scatter

根进程将数据均等拆分为 N 个子块 $\boldsymbol{X} \to (\boldsymbol{X}_1, \boldsymbol{X}_2, \cdots, \boldsymbol{X}_N)$，第 j 块发送至设备 j；各设备接收的数据块互不相同。

图 《大语言模型：训练、精调与对齐》图 9.5；第 9.1.5 节

收集与全收集

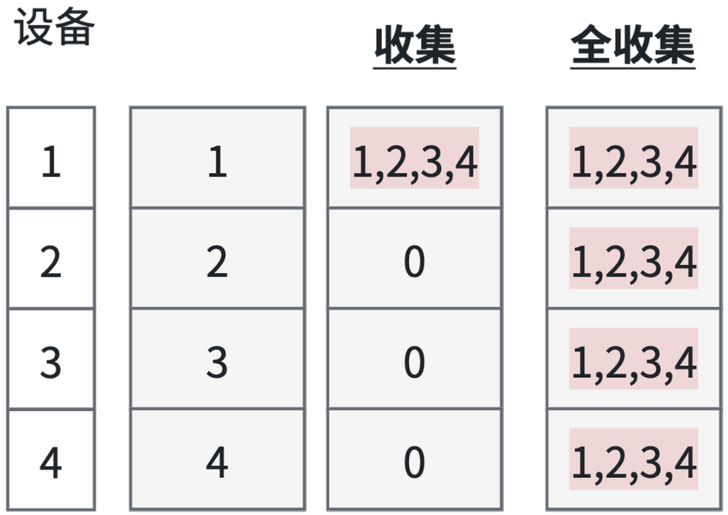


图 9.5(c)　收集和全收集

$$\boldsymbol{X} \leftarrow \texttt{Gather}(\boldsymbol{X}_1, \boldsymbol{X}_2, \cdots, \boldsymbol{X}_N)$$

$$\boldsymbol{X} \leftarrow [\boldsymbol{X}_1, \boldsymbol{X}_2, \cdots, \boldsymbol{X}_N], \quad \boldsymbol{X} \in \mathbb{R}^d$$

Gather 把 N 个数据块汇聚到根进程；All-Gather 让每个设备都得到全部数据块并按维度拼接。

收集 Gather

散播的逆操作

各设备把本地块发往根进程 i，根进程组合出完整数据集。

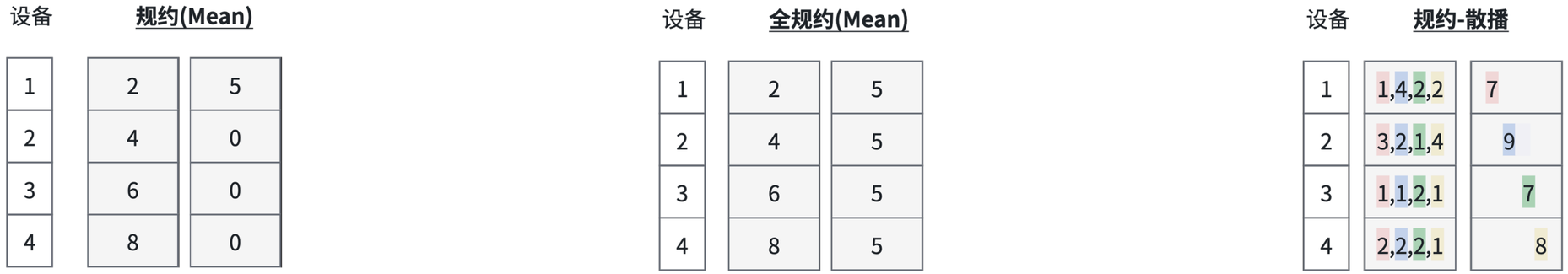
全收集 All-Gather

人人都有完整数据

设备 i 持有 $\boldsymbol{X}_i \in \mathbb{R}^{d/N}$，全收集后所有设备都获得全部数据块。

依据 《大语言模型：训练、精调与对齐》第 9.1.5 节

规约类原语



(a) 规约（以平均为例）　(b) 全规约（以平均为例）　(c) 规约-散播

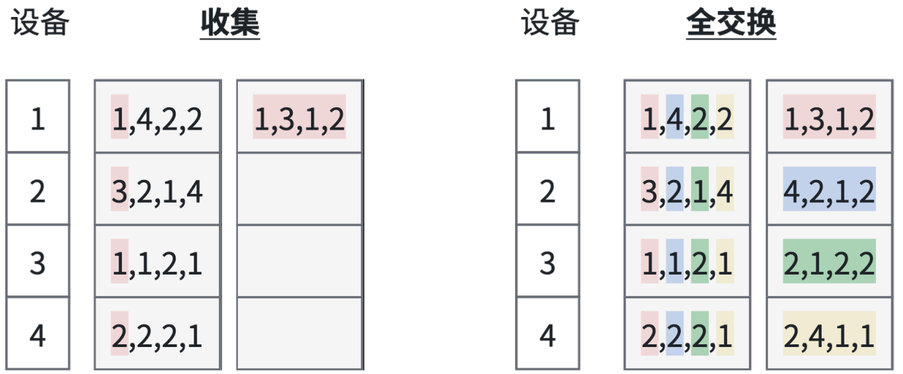
图 9.6　分布式集合通信-2

$$\tilde{\boldsymbol{X}} = \mathrm{op}(\boldsymbol{X}_1, \boldsymbol{X}_2, \cdots, \boldsymbol{X}_N), \quad \tilde{\boldsymbol{X}} \in \mathbb{R}^d$$

$$\hat{\boldsymbol{X}}^{(j)} = \mathrm{op}(\boldsymbol{X}_1^{(j)}, \boldsymbol{X}_2^{(j)}, \cdots, \boldsymbol{X}_N^{(j)}), \quad \hat{\boldsymbol{X}}^{(j)} \in \mathbb{R}^{d/N}$$

op 表示规约运算符，如求和、求平均、取最大值（如 Mean）。式 1 为 Reduce，结果存于根设备；式 2 为 Reduce-Scatter，设备 j 获得第 j 个结果块。

全交换



(a) 收集　　(b) 全交换

图 9.7　分布式集合通信-3

- 通信组内 N 个设备各自将数据划分为 N 个块，数据块大小可以不一致；设备 i 的块 $\boldsymbol{X}_i^{(j)}$ 发送至设备 j。
- 操作完成后，设备 j 接收来自所有设备的第 j 块数据 $\{\boldsymbol{X}_1^{(j)}, \boldsymbol{X}_2^{(j)}, \cdots, \boldsymbol{X}_N^{(j)}\}$。
- 在大语言模型中，全交换常用于专家并行的派发-合并过程，此时面临交换数据量不均导致的负载均衡难题。

图 《大语言模型：训练、精调与对齐》图 9.7；第 9.1.5 节

全交换与全收集的区别

朴素实现：轮流当根

各设备轮流作为根节点，依次收集来自其他设备的不同数据。每轮单次收集中，设备角色互斥（一个接收、多个发送），需同步完成 N 轮。

N 轮串行，时延随设备数线性增长。

高效实现：一个周期完成

在一个集合通信周期内完成所有数据交换，该周期内每个设备同时作为发送方和接收方。

发送与接收并行，链路利用率显著提高。

操作	发送逻辑	结果
All-Gather	广播：每个设备向所有设备发送**相同**数据	所有设备持有完整副本
All-to-All	散播：每个设备向不同目标发送**不同**数据	每个设备持有各来源的对应块

这条区别解释了两种原语在训练中的分工：All-Gather 用于“补齐”被切分的参数或激活，All-to-All 用于“换手”——把词元送到持有其专家的设备上。

9.2

混合并行策略

核心挑战在于执行远超单个设备显存容量的计算任务，同时高效利用集群资源；四种并行维度彼此正交，可以组合成 4D 并行。

本节内容

9.2.1 数据并行

9.2.2 张量并行

9.2.3 流水线并行

9.2.4 上下文并行

9.2.5 本节小结

四种并行方式

示例使用 2 个计算设备和 4 个数据样本，通过从模型参数和数据两个维度进行不同方式的切分，实现了不同的并行方法。

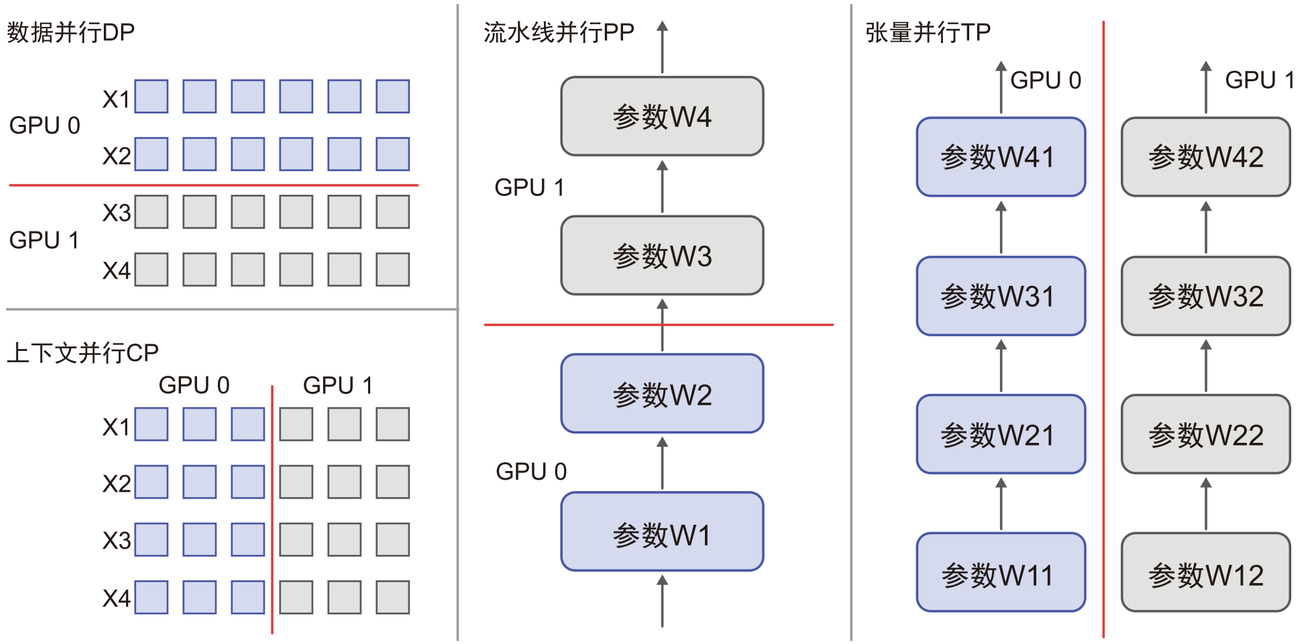


图 9.8　合并行策略（4D 并行）示例

左上

数据并行：样本均分，参数完整复制。

中间

流水线并行：按层深度切分，参数按层分属两卡。

左下

上下文并行：按序列长度切分，各自计算块注意力。

最右

张量并行：参数矩阵切分，每卡只算一部分。

切分的代价

分布式训练的核心目标是在有限的计算资源下，于单位时间内处理更多的数据量。而模型切分的主要目的，是将庞大的计算模型分解以适应有限的 GPU 显存，从而避免因显存不足而无法运行。

> **关键** 模型切分本身并不会减少训练任务的总计算量；相反，它会引入额外的通信、存储和计算开销。

收益

显存可容纳

把参数、梯度、优化器状态或激活值分摊到多张卡上，使单卡放不下的模型可以训练。

代价 1

额外通信

切分边界上需要交换中间结果或梯度，通信可能让计算单元空转。

代价 2

额外存储与计算

通信需要缓冲区，某些切分方式还会带来冗余计算与参数收集开销。

Llama-3 的 4D 并行

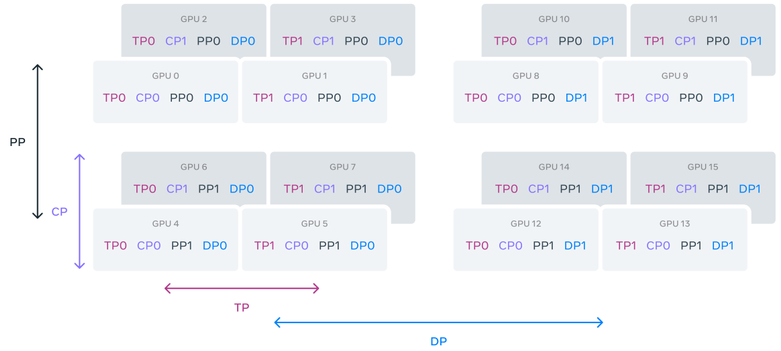


图 9.9　Llama-3 的 4D 并行训练策略

当每个维度的并行切分因子均为 2 时，四个维度的并行策略组合最多可分布在 16 个 GPU 上，即 $2^4 = 16$。

$$N_{\text{GPU}} = N_{\text{DP}} \times N_{\text{TP}} \times N_{\text{PP}} \times N_{\text{CP}} = 2 \times 2 \times 2 \times 2 = 16$$

各维度的切分因子相乘得到总设备数。通过灵活配置不同维度的并行策略，可以在混合并行框架下实现高效训练。

混合并行的核心目标在于提升吞吐量和设备利用率：通信密集的维度放节点内，通信稀疏的维度跨节点。

图 《大语言模型：训练、精调与对齐》图 9.9；第 9.2 节

数据并行

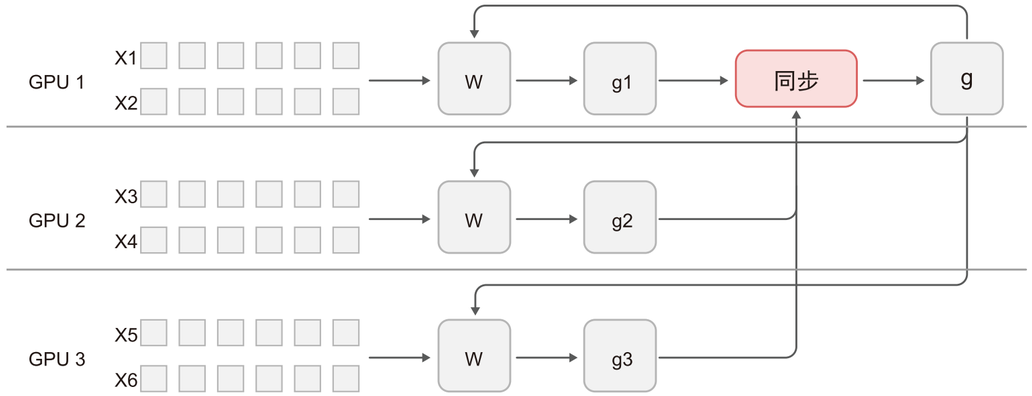


图 9.10　数据并行的梯度更新流程

数据并行在多个计算设备上加载相同参数的模型副本，每个设备处理不同批次的输入数据，独立计算梯度。

- 输入数据集 $\{\boldsymbol{x}_1, \boldsymbol{x}_2, \cdots, \boldsymbol{x}_6\}$ 被分配到 3 个 GPU 上，每个 GPU 上的模型参数 $\boldsymbol{W}$ 初始相同。
- 各设备独立前向计算损失，再根据独立的损失进行梯度反向传播。
- 随后规约各设备的梯度，同步更新参数，保持所有设备上的参数一致。

图中指定 GPU0 作为参数服务器进行梯度同步。通过增加计算设备，数据并行可以并行处理更多数据，但会引入梯度同步所需的通信开销。

图 《大语言模型：训练、精调与对齐》图 9.10；第 9.2.1 节

梯度同步的时机

梯度同步可以在整个反向传播过程完成后一次性进行，也可以在反向传播过程中分层进行。

批量更新

反向传播全部完成后统一同步一次梯度。

通信次数少，可以合并成大消息、充分利用带宽。

代价：需要保存完整梯度，内存占用高。

立即更新

每一层梯度算完就立即发起同步，与后续层的反向计算重叠。

内存占用较低，通信可被计算掩盖。

代价：通信次数多，小消息的启动开销占比高。

> **权衡** 批量更新通信次数少但内存占用高；立即更新通信次数多但内存占用较低。选择取决于带宽与显存哪一侧更紧张。

两种梯度同步机制

机制 1

参数服务器（Parameter Server）

指定一个设备作为服务器。其他设备将计算的梯度发送给服务器；服务器聚合所有梯度，通常计算均值 $\bar{g}$，然后将聚合结果 $\bar{g}$ 广播回所有设备。

服务器收集梯度时，发送完梯度的设备可能处于空闲等待状态，降低了资源利用率。

机制 2

All-Reduce 集合通信

所有设备通过高效的集合通信操作直接交换梯度数据，最终在每个设备上得到相同的聚合梯度 $\bar{\boldsymbol{g}}$。

All-Reduce 通常是同步的：所有设备必须完成当前计算并参与通信，较快的设备要等待较慢的设备，称为“拖尾”现象。

$$\bar{\boldsymbol{g}} = \texttt{All-Reduce}\left(\boldsymbol{g}_0, \boldsymbol{g}_1, \cdots, \boldsymbol{g}_{N-1}\right) = \frac{1}{N}\sum_{i=0}^{N-1}\boldsymbol{g}_i$$

$\boldsymbol{g}_i$ 为设备 i 在本地批次上算出的梯度；全规约后每张卡都持有相同的均值梯度，从而保证参数一致。

依据 《大语言模型：训练、精调与对齐》第 9.2.1 节

异步数据并行

同步模式要求所有设备等待梯度全规约完成后再同步更新参数。异步数据并行则不同：各节点完成本地梯度计算后，无需等待其他节点，立即使用这些梯度更新全局模型参数。

$$\boldsymbol{W}_{t+1} = \boldsymbol{W}_t - \eta \boldsymbol{g}_A(\boldsymbol{W}_{t-\tau}), \quad \tau \geqslant 0$$

节点 A 基于版本 $\boldsymbol{W}_t$ 计算梯度时，参数服务器可能已被节点 B 更新为 $\boldsymbol{W}_{t+1}$，导致节点 A 的梯度相对于最新参数已过时。τ 表示滞后步数。

优势

没有等待

避免快速节点等待慢速节点，提高硬件利用率。

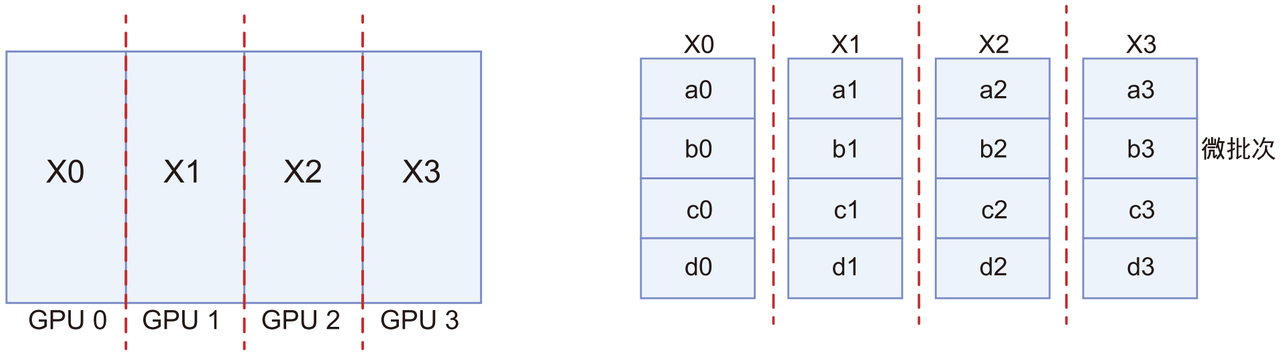
劣势

收敛风险

过时梯度可能导致训练不稳定和收敛困难；参数服务器可能成为通信瓶颈（尤其单服务器架构）。

随着 GPU 数量增加，参数版本差异加剧，可能显著影响模型收敛性。

环形全规约



(a) 数据并行　　(b) 微批次数据并行

图 9.11　环形全规约数据划分示意图

- 共有 n 个 GPU，每个 GPU j 持有全局数据的一个互斥分块 $\boldsymbol{X}_j \in \mathbb{R}^{b\times d}$，$b$ 为局部批次大小。
- 每个分块可进一步划分为 k 个微批次 $\boldsymbol{X}_{ij} \in \mathbb{R}^{b'\times d}$，其中 $b' = b/k$。
- 通信采用环形拓扑：每个 GPU i 将数据发送给后继 $(i+1)\%n$，并从前驱 $(i-1)\%n$ 接收；GPU $n-1$ 的后继是 GPU 0。

环形全规约阶段一

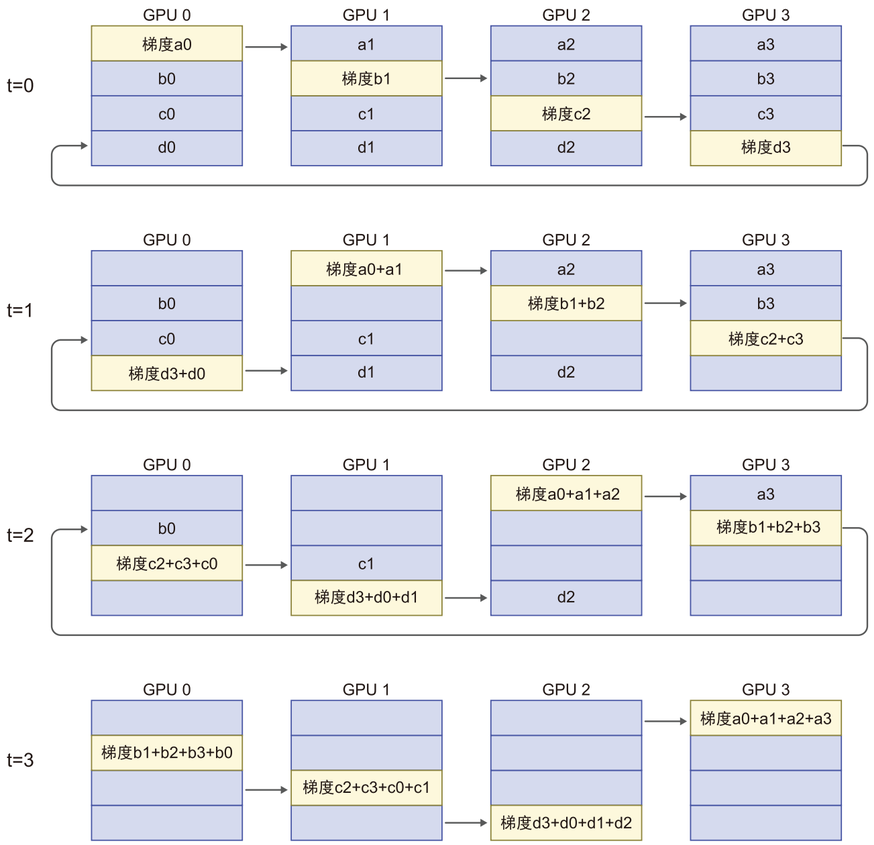


图 9.12　环形全规约：规约-散播阶段

- 目标：在 $n-1$ 步内，每个 GPU 最终持有其中一个微批次数据块梯度的总和。
- 每一步，每个 GPU i 计算当前持有数据的梯度，把梯度分块（或分块和）发送给后继 GPU。
- 同时从前驱接收一个分块，并与本地对应的分块累加。
- 经过 $n-1$ 步后，GPU i 存储了全局梯度总和的一个完整分块 $\tilde{G}_i$。

$$\tilde{G}_i = \sum_{j=1}^{n} G_{i,j}$$

$\tilde{G}_i$ 为第 i 个梯度分块在所有 GPU 上的总和，最终只落在 GPU i 上；其余分块的求和结果落在其余 GPU 上。

图 《大语言模型：训练、精调与对齐》图 9.12；第 9.2.1 节

环形全规约阶段二

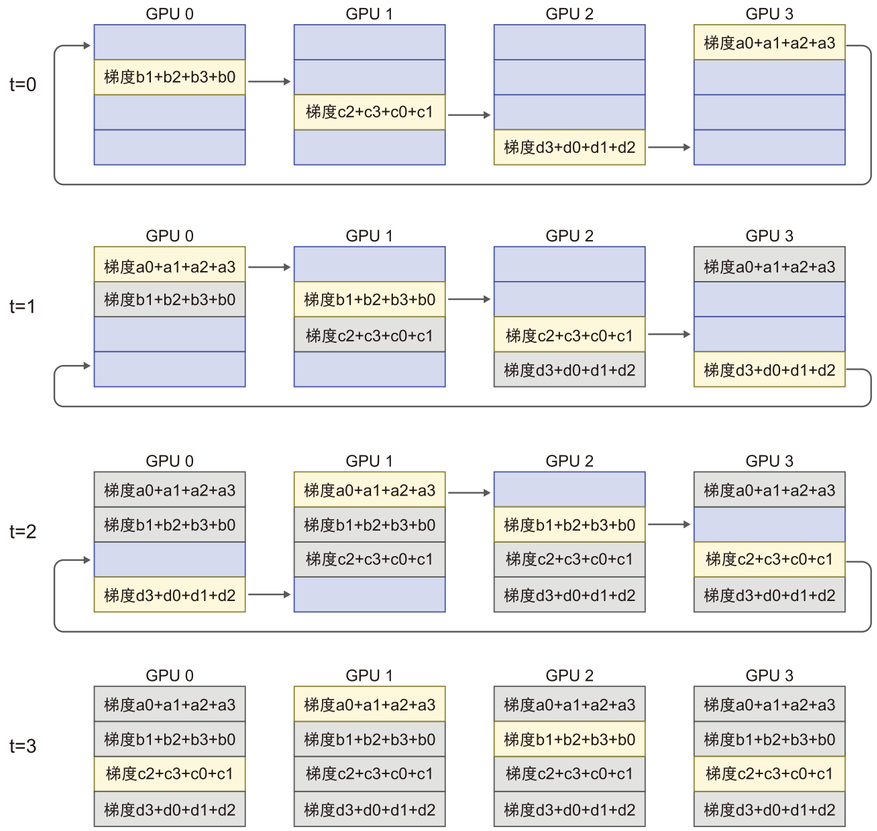


图 9.13　环形全规约：全收集阶段

- 目标：在 $n-1$ 步内，把规约-散播阶段产生的 n 个分块总和广播到所有 GPU。
- 每一步，每个 GPU i 将当前持有的分块发送给后继，同时从前驱接收一个尚未持有的分块。
- 经过 $n-1$ 步，每个 GPU 都存储了所有 n 个分块，即完整的全局梯度总和，实现了梯度一致性。

两个阶段合计 $2(n-1)$ 步，每步传输 $O(N/n)$ 数据：总传输量 $O(N)$，且所有链路并行工作。

$$\boldsymbol{G}_{\text{total}} = \sum_{i=0}^{n-1} \tilde{G}_i = \sum_{j=0}^{n-1} \boldsymbol{g}_j$$

全收集结束后，每张卡上的 $\boldsymbol{G}_{\text{total}}$ 相同，正是各设备梯度的总和（按需再除以 n 得到均值）。

图 《大语言模型：训练、精调与对齐》图 9.13；第 9.2.1 节

4 个 GPU 的环形全规约

假设全局批次被划分为 4 个互斥分块 $\boldsymbol{X}_0, \boldsymbol{X}_1, \boldsymbol{X}_2, \boldsymbol{X}_3$，分别由 GPU0–GPU3 持有。每个 GPU 在其微批次分块（如 $\boldsymbol{X}_0$ 分为 a_0, b_0, c_0, d_0）上计算梯度，一步操作后各 GPU 得到对应的 $G_{a0}, G_{b1}, G_{c2}, G_{d3}$。



阶段	步数	阶段结束时的状态
规约-散播	4 − 1 = 3	GPU0 持有 G_b 的总和，GPU1 持有 G_c 的总和，GPU2 持有 G_d 的总和，GPU3 持有 G_a 的总和
全收集	3	每张卡都获得 $G_{\text{total}} = G_a + G_b + G_c + G_d$ 的所有分块

分块与持有者的错位（GPU0 拿 G_b 而非 G_a）来自环形传递的步进：每步发送后，当前块归属于后继链路上的位置。

依据 《大语言模型：训练、精调与对齐》第 9.2.1 节

环形与树形的对比

全规约的实现主要有两种形式：环形全规约与树形全规约。两者总数据传输量均为 $O(N)$（N 为梯度总量），但通信模式和时间复杂度存在显著差异。

环形全规约

每个节点只与相邻节点通信。通信步数 $O(n)$，每步传输 $O(N/n)$。

优势：所有链路并行传输，带宽利用率高，适合大规模集群。缺点：延迟随节点数线性增长。

树形全规约

采用树状拓扑（如二叉树），通信步数 $O(\log n)$，时间复杂度更低。

缺点：高层级节点（尤其根节点）的链路带宽压力集中，容易成为瓶颈并引发网络拥塞。

形式	通信步数	单步数据量	瓶颈位置
环形	$O(n)$	$O(N/n)$	延迟随节点数增长
树形	$O(\log n)$	逐层变化	根节点与高层链路的带宽

因此，尽管树形全规约的步数复杂度更优，但在大规模节点场景下，环形全规约凭借均衡的带宽利用率和可扩展性，往往能获得更优的实际吞吐量。

依据 《大语言模型：训练、精调与对齐》第 9.2.1 节

张量并行

当模型参数规模过大，导致单个 GPU 的显存无法容纳所有参数时，或者仅为了分解计算规模，需要将模型参数切分并部署到多个 GPU 上存储。张量并行的核心思想，是将神经网络层中的权重矩阵进行切分，使每个设备仅需存储部分权重。

收益

显存按比例下降

每个设备只存部分权重，相应地，所需存储的激活值和优化器状态量也随之减少。

代价

通信随粒度上升

随着切分粒度增加（使用更多设备并行），设备间的通信开销也会显著增大。

本质

用通信换显存

实际加速效果受限于设备间的通信带宽和延迟，是以增加通信开销换取降低单设备显存的优化策略。

下面先分析矩阵乘法中的不同行列切分策略，进而探讨 Transformer 架构下的张量并行设计。

矩阵乘法的切分

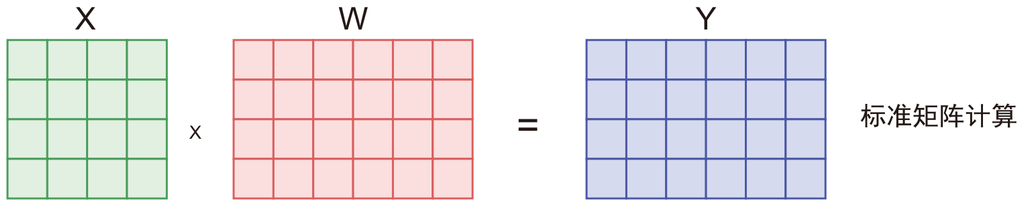


图 9.14　标准矩阵乘法示意图

$$\boldsymbol{Y} = \boldsymbol{X}\boldsymbol{W}, \quad \boldsymbol{Y} \in \mathbb{R}^{2N \times 2R} \tag{9-1}$$

在张量并行组内使用 n 个设备，为简化说明以 $n = 2$ 为例。给定权重矩阵 $\boldsymbol{W} \in \mathbb{R}^{2D \times 2R}$ 和输入矩阵 $\boldsymbol{X} \in \mathbb{R}^{2N \times 2D}$。

当 $\boldsymbol{W}$ 超出单个设备的显存容量时，有两种自然的切法：按列切成左右两块，或按行切成上下两块。两种切法在前向与反向中需要的通信位置恰好相反。

列切分

$\boldsymbol{W} = [\boldsymbol{A}_1, \boldsymbol{A}_2]$，每卡持有一部分**输出列**；前向各算一半、反向需规约输入梯度。

行切分

$\boldsymbol{W} = [\boldsymbol{B}_1; \boldsymbol{B}_2]$，每卡持有一部分**输入行**；前向需规约输出、反向各算一半。

图 《大语言模型：训练、精调与对齐》图 9.14；式 (9-1)

列切分

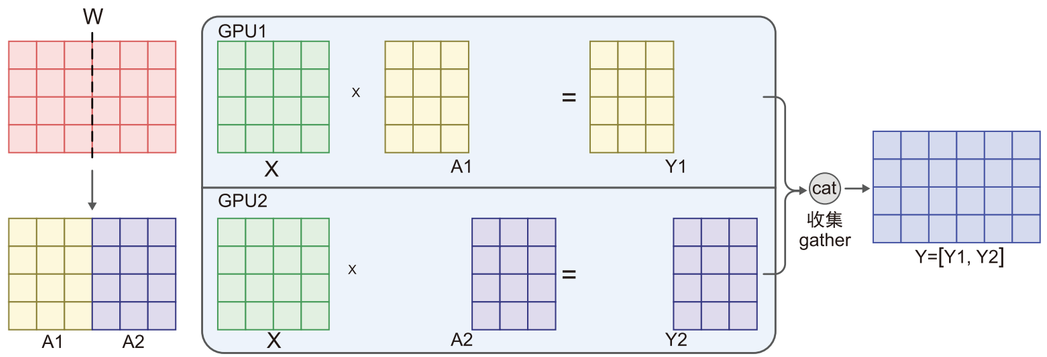


图 9.15　权重矩阵列切分的张量并行示意图

$$[\boldsymbol{A}_1, \boldsymbol{A}_2] = \texttt{Split_col}(\boldsymbol{W}) \tag{9-2}$$

$$\boldsymbol{Y} = \boldsymbol{X}[\boldsymbol{A}_1, \boldsymbol{A}_2] = [\boldsymbol{Y}_1, \boldsymbol{Y}_2] \tag{9-3}$$

$$\boldsymbol{Y}_1 = \boldsymbol{X}\boldsymbol{A}_1, \qquad \boldsymbol{Y}_2 = \boldsymbol{X}\boldsymbol{A}_2$$

$\boldsymbol{A}_1, \boldsymbol{A}_2 \in \mathbb{R}^{2D\times R}$，分别存储于 GPU1 和 GPU2；$\boldsymbol{Y}_1, \boldsymbol{Y}_2 \in \mathbb{R}^{2N\times R}$。

- 每块 GPU 需要完整的输入 $\boldsymbol{X}$，因此需先将 $\boldsymbol{X}$ 广播至所有设备。
- 最终通过收集操作得到完整输出 $\boldsymbol{Y} \in \mathbb{R}^{2N\times 2R}$。

列切分没有把输入切开，所以每张卡都要读整份 $\boldsymbol{X}$；它的通信集中在前向的广播与反向的规约。

图 《大语言模型：训练、精调与对齐》图 9.15；式 (9-2)、(9-3)

列切分的反向传播

$$\mathrm{d}\boldsymbol{A}_1 = \boldsymbol{X}^T \mathrm{d}\boldsymbol{Y}_1, \quad \mathrm{d}\boldsymbol{A}_2 = \boldsymbol{X}^T \mathrm{d}\boldsymbol{Y}_2 \tag{9-4}$$

$$\mathrm{d}\boldsymbol{X} = \mathrm{d}\boldsymbol{Y}_1 \boldsymbol{A}_1^T + \mathrm{d}\boldsymbol{Y}_2 \boldsymbol{A}_2^T \tag{9-5}$$

$$= \texttt{All-Reduce}\left(\mathrm{d}\boldsymbol{Y}_1 \boldsymbol{A}_1^T,\ \mathrm{d}\boldsymbol{Y}_2 \boldsymbol{A}_2^T\right) \tag{9-6}$$

给定损失对输出的梯度 $\mathrm{d}\boldsymbol{Y} \in \mathbb{R}^{2N\times 2R}$，按列切分为 $\mathrm{d}\boldsymbol{Y}_1, \mathrm{d}\boldsymbol{Y}_2 \in \mathbb{R}^{2N\times R}$。

第 1 步 · 权重梯度

本地独立计算

每张卡用自己那份 $\mathrm{d}\boldsymbol{Y}_i$ 与本地的 $\boldsymbol{A}_i$、完整的 $\boldsymbol{X}^T$ 相乘，得到 $\mathrm{d}\boldsymbol{A}_1, \mathrm{d}\boldsymbol{A}_2 \in \mathbb{R}^{2D\times R}$，无需通信。

第 2 步 · 输入梯度

跨设备全规约

完整输入梯度 $\mathrm{d}\boldsymbol{X} \in \mathbb{R}^{2N\times 2D}$ 是两份局部贡献之和，通过 All-Reduce 求和得到。

式 (9-5) 拆成两项正是因为输出被按列切开：每张卡只掌握一部分 $\mathrm{d}\boldsymbol{Y}$，对应的输入梯度也只是部分和。

依据 《大语言模型：训练、精调与对齐》第 9.2.2 节（式 (9-4)–(9-6)）

行切分

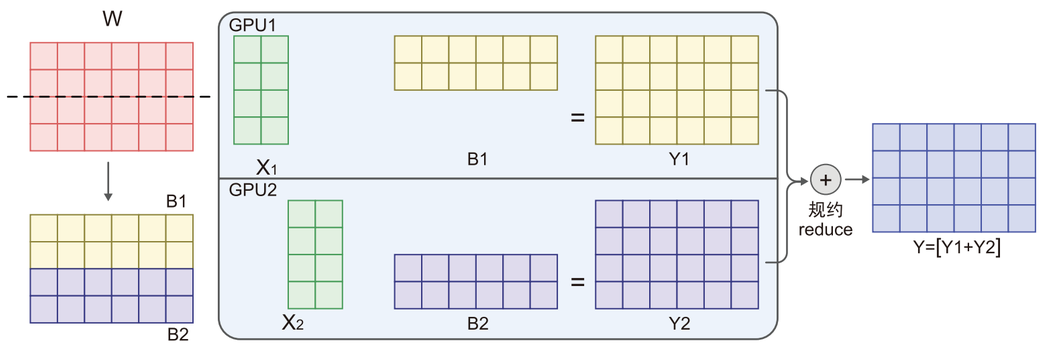


图 9.16　权重矩阵行切分的张量并行示意图

$$\begin{bmatrix} \boldsymbol{B}_1 \\ \hline \boldsymbol{B}_2 \end{bmatrix} \leftarrow \texttt{Split_row}(\boldsymbol{W}) \tag{9-7}$$

$$[\boldsymbol{X}_1, \boldsymbol{X}_2] \leftarrow \texttt{Split_col}(\boldsymbol{X}) \tag{9-8}$$

$$\boldsymbol{Y} = [\boldsymbol{X}_1, \boldsymbol{X}_2] \begin{bmatrix} \boldsymbol{B}_1 \\ \hline \boldsymbol{B}_2 \end{bmatrix} = \underbrace{\boldsymbol{X}_1 \boldsymbol{B}_1}_{\boldsymbol{Y}_1} + \underbrace{\boldsymbol{X}_2 \boldsymbol{B}_2}_{\boldsymbol{Y}_2} \tag{9-9}$$

$\boldsymbol{B}_1 \in \mathbb{R}^{D \times 2R}$ 存于 GPU1，$\boldsymbol{B}_2 \in \mathbb{R}^{D \times 2R}$ 存于 GPU2；$\boldsymbol{X}_1, \boldsymbol{X}_2 \in \mathbb{R}^{2N \times D}$。

- $\boldsymbol{Y}_1, \boldsymbol{Y}_2 \in \mathbb{R}^{2N \times 2R}$ 分别位于两张卡，需通过 $\texttt{All-Reduce}(\cdot)$ 求和得到完整输出 $\boldsymbol{Y} \leftarrow \boldsymbol{Y}_1 + \boldsymbol{Y}_2$。
- 行切分把输入也按列切开了，因此每张卡只需读一半 $\boldsymbol{X}$，代价是前向必须做一次规约。

图 《大语言模型：训练、精调与对齐》图 9.16；式 (9-7)–(9-9)

行切分的反向传播

$$\mathrm{d}\boldsymbol{B}_1 = \boldsymbol{X}_1^T \mathrm{d}\boldsymbol{Y}, \quad \mathrm{d}\boldsymbol{B}_2 = \boldsymbol{X}_2^T \mathrm{d}\boldsymbol{Y} \tag{9-10}$$

$$\mathrm{d}\boldsymbol{X}_1 = \mathrm{d}\boldsymbol{Y} \boldsymbol{B}_1^T, \quad \mathrm{d}\boldsymbol{X}_2 = \mathrm{d}\boldsymbol{Y} \boldsymbol{B}_2^T \tag{9-11}$$

$$\mathrm{d}\boldsymbol{X} \leftarrow [\mathrm{d}\boldsymbol{X}_1, \mathrm{d}\boldsymbol{X}_2] \tag{9-12}$$

$\mathrm{d}\boldsymbol{Y} \in \mathbb{R}^{2N \times 2R}$ 需先广播至所有设备；$\mathrm{d}\boldsymbol{B}_1, \mathrm{d}\boldsymbol{B}_2 \in \mathbb{R}^{D \times 2R}$，$\mathrm{d}\boldsymbol{X}_1, \mathrm{d}\boldsymbol{X}_2 \in \mathbb{R}^{2N \times D}$。

广播

$\mathrm{d}\boldsymbol{Y}$ 发到所有卡

→

权重梯度

各卡独立算 $\mathrm{d}\boldsymbol{B}_i$ (9-10)

→

输入梯度

各卡独立算 $\mathrm{d}\boldsymbol{X}_i$ (9-11)

→

拼接

`Gather` 得完整 $\mathrm{d}\boldsymbol{X}$ (9-12)

与列并行对照：列并行的通信在反向的输入梯度上，行并行的通信在前向的输出求和与反向的输出梯度广播上。这一“互补”关系正是 FFN 与注意力层能各取所长的基础。

依据 《大语言模型：训练、精调与对齐》第 9.2.2 节（式 (9-10)–(9-12)）

FFN 层的张量并行

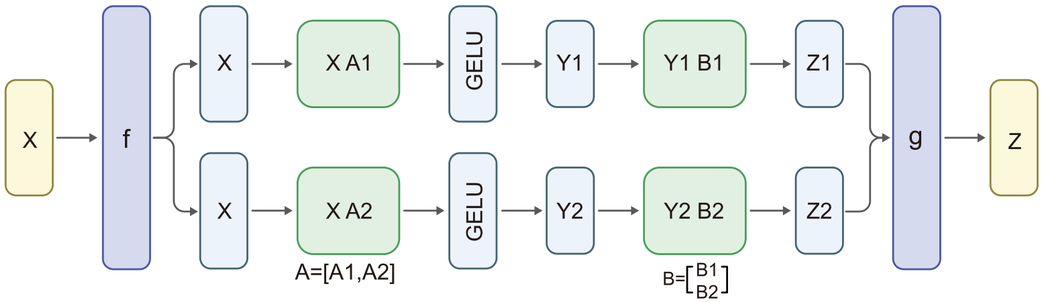


图 9.17　MLP 层的张量并行切分方式

$$\boldsymbol{Y} = \sigma(\boldsymbol{X}\boldsymbol{W}_1 + \boldsymbol{B}_1)\boldsymbol{W}_2 + \boldsymbol{B}_2 \tag{9-13}$$

$$[\boldsymbol{Y}_1, \boldsymbol{Y}_2] = [\mathtt{GELU}(\boldsymbol{X}\boldsymbol{A}_1), \mathtt{GELU}(\boldsymbol{X}\boldsymbol{A}_2)] \tag{9-15}$$

$$\boldsymbol{Z} = [\boldsymbol{Y}_1, \boldsymbol{Y}_2]\begin{bmatrix}\boldsymbol{B}_1 \\ \boldsymbol{B}_2\end{bmatrix} = \boldsymbol{Y}_1\boldsymbol{B}_1 + \boldsymbol{Y}_2\boldsymbol{B}_2 \tag{9-16}$$

$\boldsymbol{W}_1 \in \mathbb{R}^{D\times 4D}$ 列切分为 $\boldsymbol{A}_1, \boldsymbol{A}_2 \in \mathbb{R}^{D\times 2D}$；$\boldsymbol{W}_2 \in \mathbb{R}^{4D\times D}$ 行切分为 $\boldsymbol{B}_1, \boldsymbol{B}_2 \in \mathbb{R}^{2D\times D}$。

为什么必须这样配？若对 $\boldsymbol{W}_1$ 采用行切分，由于激活函数的非线性特性，先求和再激活与先激活再求和并不相等：

$$\mathtt{GELU}(\boldsymbol{X}_1\boldsymbol{W}_{11} + \boldsymbol{X}_2\boldsymbol{W}_{12}) \neq \mathtt{GELU}(\boldsymbol{X}_1\boldsymbol{W}_{11}) + \mathtt{GELU}(\boldsymbol{X}_2\boldsymbol{W}_{12}) \tag{9-14}$$

图 《大语言模型：训练、精调与对齐》图 9.17；式 (9-13)–(9-16)

FFN 张量并行的推导

$$\boldsymbol{Z} = \boldsymbol{Y}_1\boldsymbol{B}_1 + \boldsymbol{Y}_2\boldsymbol{B}_2 = \boldsymbol{Z}_1 + \boldsymbol{Z}_2 \tag{9-17}$$

$$\boldsymbol{Z} = \texttt{All-Reduce}(\boldsymbol{Z}_1, \boldsymbol{Z}_2) \tag{9-18}$$

$\boldsymbol{Y}_1, \boldsymbol{Y}_2 \in \mathbb{R}^{2N\times 2D}$，$\boldsymbol{Z}, \boldsymbol{Z}_1, \boldsymbol{Z}_2 \in \mathbb{R}^{2N\times D}$。式 (9-17) 是局部矩阵乘，式 (9-18) 是前向必须的一次规约。

通信账

一次前向 + 一次反向

前向需要一次 All-Reduce 求出 $\boldsymbol{Z}$；反向传播中，计算输入梯度 $\mathbf{d}\boldsymbol{X}$ 时需再次执行 All-Reduce。

存储账

激活值冗余

在计算梯度 $\mathbf{d}\boldsymbol{A}_1, \mathbf{d}\boldsymbol{A}_2$ 时，各 GPU 前向可能存储了 $\boldsymbol{X}$ 的值用于反向计算，会导致一定的数据占用冗余。

这一对“列切分 + 行切分”的组合是 Megatron-LM 张量并行的核心模式：在层内部完成切分与计算，只在层的出口处做一次规约，把通信压到最少。

依据 《大语言模型：训练、精调与对齐》第 9.2.2 节（式 (9-17)、(9-18)）

注意力层的张量并行

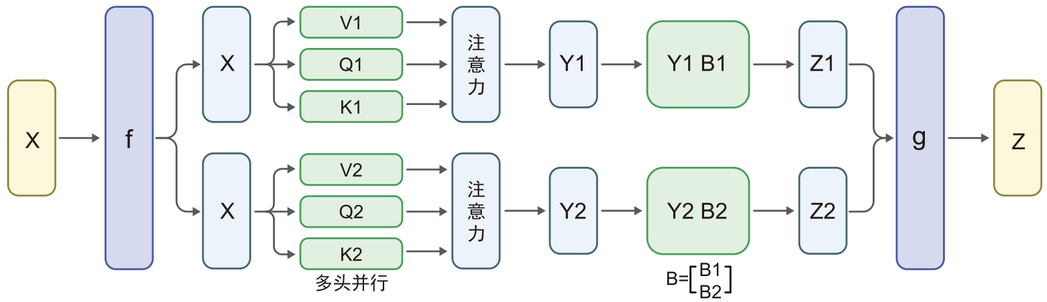


图 9.18　自注意力层头并行策略

$$\boldsymbol{W}_q^{(i)}, \boldsymbol{W}_k^{(i)}, \boldsymbol{W}_v^{(i)} \in \mathbb{R}^{d \times d'} \tag{9-19}$$

$$\boldsymbol{Z}^{(i)} = \texttt{Attn}\left(\boldsymbol{X}\boldsymbol{W}_q^{(i)}, \boldsymbol{X}\boldsymbol{W}_k^{(i)}, \boldsymbol{X}\boldsymbol{W}_v^{(i)}\right) \tag{9-20}$$

$d' = d/N$，N 为设备数，设注意力头数与设备数相同；$\boldsymbol{X}$ 为完整输入。

- $\boldsymbol{W}_q, \boldsymbol{W}_k, \boldsymbol{W}_v$ 按列切分，每个设备仅处理部分注意力头。
- 输出投影矩阵 $\boldsymbol{W}_o$ 按行切分，最终通过 All-Reduce 规约结果。

这一方案称为头并行。它避免了先计算完整 $\matQKV$ 矩阵再切分散播的通信开销；对于分组查询注意力，若使用键值头共享，则需在组内同步 $\boldsymbol{W}_k^{(i)}, \boldsymbol{W}_v^{(i)}$ 参数。

图 《大语言模型：训练、精调与对齐》图 9.18；式 (9-19)、(9-20)

其他层的张量并行

归一化层

参数量小，两种都行

可以在每个设备上冗余存储，也可以沿特征维度切分参数。

输入来自列并行层时，求均值与方差需设备间同步；输入来自行并行层时，整个归一化可在各设备内独立完成。

词嵌入层

词表大，必须切

（1）**词表并行**：将词表划分为若干子集分配给不同设备，每个设备存储完整特征维度的部分词向量。

（2）**特征维度并行**：将词向量沿特征维度切分，各设备存储所有词元的部分特征。

语言输出层

计算量大，切法有别

同样可按词表维度或特征维度划分，但投影矩阵 $\boldsymbol{W}_{\text{output}} \in \mathbb{R}^{d\times|\mathcal{V}|}$ 在前后向均涉及大规模矩阵计算，且需接 Softmax。

特征维度划分较易实现；词表划分更复杂，需特殊处理如并行交叉熵损失计算。

> **规模感** 当 $d = 4096$、$|\mathcal{V}| = 128\,000$ 时，输出层单个权重矩阵就有约 5.2 亿个元素——它本身就是一块需要被切分的显存。

词嵌入层通过词元索引获取向量，而输出层要对所有词元做完整的矩阵乘法，这就是两者切分策略不能简单照搬的原因。

张量并行小结

张量并行的核心特征是计算过程中需频繁通信：前向传播中，行并行的中间结果需强制规约求和；反向传播中，列并行的输入梯度需强制规约求和。

对比维度	数据并行	张量并行
通信时机	仅反向传播结束后同步梯度	前向与反向计算过程中均插入通信
通信频率	每步一次（或每层一次）	每层多次，随切分粒度上升
切分对象	数据样本	层内参数矩阵
典型部署	跨节点	节点内（NVLink / NVSwitch）

设计原则

四条约束

（1）相邻层间的切分衔接；（2）前向与反向传播的计算特性；（3）通信开销与显存占用的平衡；（4）层内的计算特性。

应用

推理与训练的差别

推理场景中，张量并行因能降低单设备显存需求、提高并行计算效率被广泛采用；但频繁通信在训练或推理系统中都可能成为效率瓶颈。

思考题与实践题

下面 8 题均为动手实现，建议先用 2–4 个进程在本机跑通，再放大到多卡。实现时不必追求与生产框架同等的性能，重点是把通信原语的语义与数值一致性验证清楚。

1 实现 Reduce-Scatter：输入各进程的梯度张量，输出按 rank 分块规约的结果，并与 All-Reduce 后切片的结果逐元素比对。

2 实现非阻塞的 All-to-All：用 `isend/irecv` 拼出不带全局屏障的交换，比较它与阻塞版本在多轮叠加下的耗时。

3 实现分布式 Adam 优化器及梯度累积更新：参数与优化器状态分区存放，验证累积 K 步后更新与逐步更新等价。

4 实现 GQA 的张量并行前向与反向：KV 头在 TP 组内共享，验证前向输出与单卡结果一致、反向梯度能正确规约。

5 实现流水线并行的 1F1B：P 个阶段、M 个微批量，画出各阶段的时隙表，标出气泡并计算气泡占比。

6 实现基于 All-Reduce 的词表并行：输出层按词表切分，前向 All-Gather、反向 Reduce-Scatter，验证与不分词的 logits 一致；再实现嵌入层词表并行，嵌入表按词表切分，前向 All-Gather 拼出各词元向量、反向 Reduce-Scatter 分回梯度。

7 实现 ZeRO-3 训练：参数、梯度、优化器状态三分区，前向 All-Gather 取参数、用完释放，报告每卡显存与通信量。

8 实现专家并行，再进一步实现计算-通信重叠：用异步的 All-to-All 与专家计算重叠，比较重叠前后的吞吐率。

推荐使用 torch.distributed 模块进行实现。

依据 《大语言模型：训练、精调与对齐》第 9 章